PYTHON ACTIVITY-BASED LEARNING PROJECT

Project Report | 10 Practical Python Case Studies

Project	Python Activity-Based Learning (ABL)
Programming Language	Python
Repository	Python-Activity-Based-Learning
GitHub Branch	main
Student Name	Siddisha Inapakolla
Roll Number	26001142
Class / Section	B.Tech CSE (AI & ML) | CSM_5
Course / Subject	Python Programming

Source: public GitHub repository — https://github.com/siddishachowdaryinapakolla-maker/Python-Activity-Based-Learning

Project Overview

This project contains ten practical Python case studies. The programs demonstrate input and output, variables, data types, arithmetic operators, conditional statements, logical operators and decision-making. Each case study includes its problem statement, original source code, sample input and output, a standard-symbol flowchart and a conclusion.

Standard Flowchart Symbols

Symbol	Meaning	Purpose
Oval	Terminator	Start or End
Parallelogram	Input / Output	Read values or display results
Rectangle	Process	Calculation or assignment
Diamond	Decision	Condition with Yes / No paths

Sample outputs are based on the sample inputs shown. Different inputs will produce different results. The source code is reproduced from the GitHub repository.

Case Study 1: Student Marks and Result Calculation

1. Problem Statement

Write a Python program to accept marks in Python, Mathematics and English. Calculate the total marks and percentage, then display PASS only when the student scores at least 35 marks in every subject; otherwise display FAIL.

2. Python Source Code

```
python_marks = int(input("Enter Python marks: "))
maths_marks = int(input("Enter Maths marks: "))
english_marks = int(input("Enter English marks: "))

total = python_marks + maths_marks + english_marks
percentage = total / 3

print("Total Marks:", total)
print("Percentage:", percentage)

if python_marks >= 35 and maths_marks >= 35 and english_marks >= 35:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

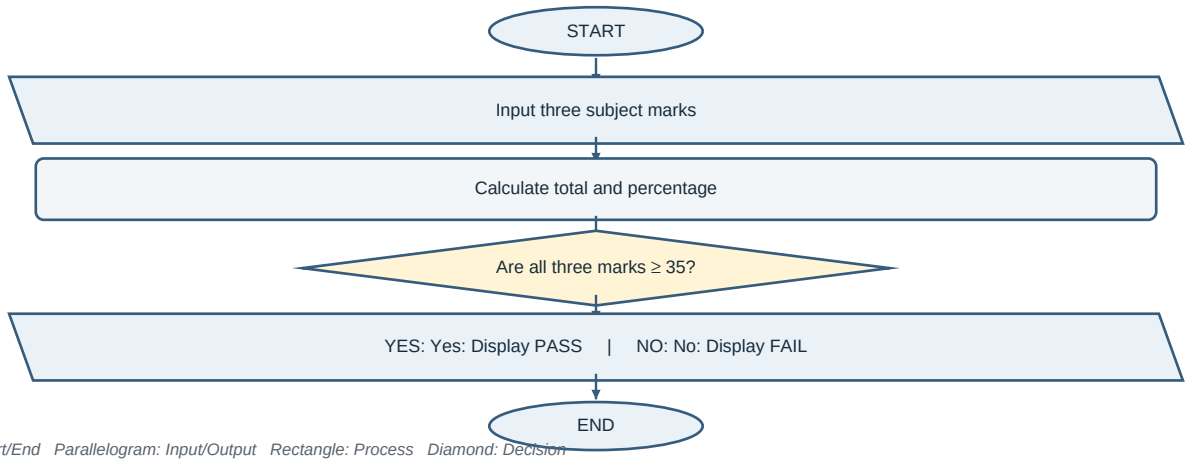
3. Sample Input

Python = 80, Maths = 75, English = 90

4. Sample Output

```
Enter Python marks: 80
Enter Maths marks: 75
Enter English marks: 90
Total Marks: 245
Percentage: 81.66666666666667
Result: PASS
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program calculates the total and average percentage and uses a logical AND condition to check whether all three subject marks meet the pass mark.

Case Study 2: Vehicle Fuel Requirement and Cost

1. Problem Statement

Write a Python program to calculate the fuel required for a journey and its total cost. Accept distance, vehicle mileage and fuel price. Perform the calculation only when all three inputs are positive.

2. Python Source Code

```
distance = float(input("Enter distance in km: "))
mileage = float(input("Enter mileage: "))
fuel_price = float(input("Enter fuel price: "))

if distance > 0 and mileage > 0 and fuel_price > 0:
    fuel = distance / mileage
    cost = fuel * fuel_price

    print("Fuel Required:", fuel, "litres")
    print("Total Fuel Cost: Rs.", cost)
else:
    print("Invalid Input")
```

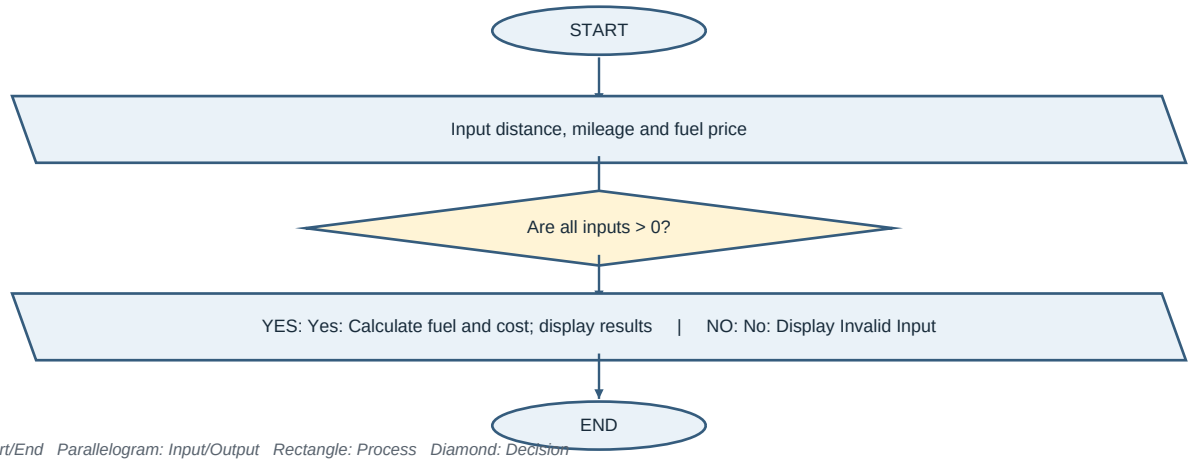
3. Sample Input

Distance = 120 km, Mileage = 15 km/litre, Fuel price = Rs. 100/litre

4. Sample Output

```
Enter distance in km: 120
Enter mileage: 15
Enter fuel price: 100
Fuel Required: 8.0 litres
Total Fuel Cost: Rs. 800.0
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program applies arithmetic operations to estimate fuel consumption and cost. A condition prevents calculations when any input is zero or negative.

Case Study 3: Production Cost and Profit or Loss

1. Problem Statement

Write a Python program to accept material cost, labour cost, machine cost and selling price. Calculate the total production cost and determine whether the business earns a profit, makes no profit or loss, or incurs a loss.

2. Python Source Code

```
material = float(input("Enter material cost: "))
labour = float(input("Enter labour cost: "))
machine = float(input("Enter machine cost: "))
selling_price = float(input("Enter selling price: "))

total_cost = material + labour + machine
profit = selling_price - total_cost

print("Total Production Cost:", total_cost)
print("Profit:", profit)

if profit > 0:
    print("Result: Profit")
elif profit == 0:
    print("Result: No Profit No Loss")
else:
    print("Result: Loss")
```

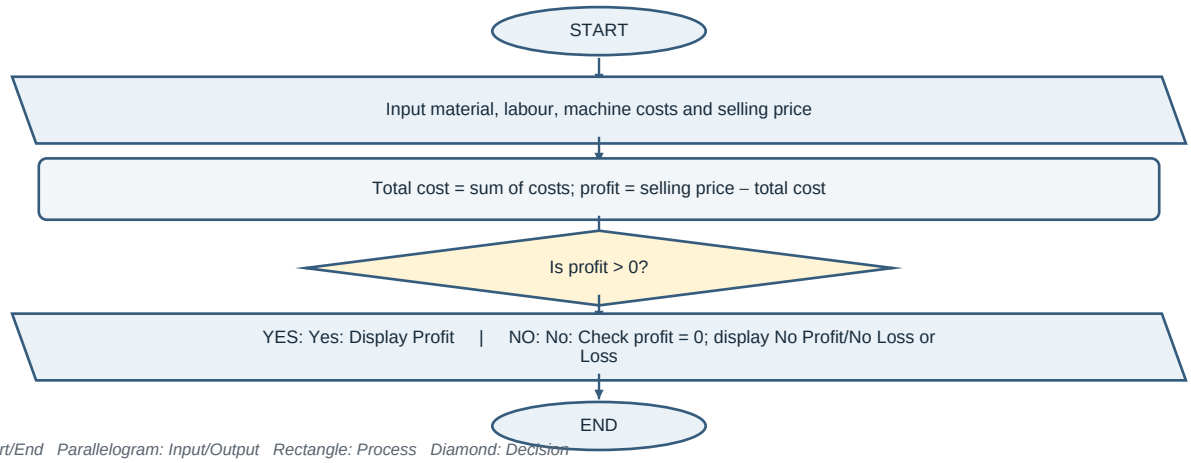
3. Sample Input

Material = Rs. 1000, Labour = Rs. 500, Machine = Rs. 300, Selling price = Rs. 2500

4. Sample Output

```
Enter material cost: 1000
Enter labour cost: 500
Enter machine cost: 300
Enter selling price: 2500
Total Production Cost: 1800.0
Profit: 700.0
Result: Profit
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program calculates total production cost and compares it with the selling price. The if-elif-else structure classifies the result as profit, no profit/no loss, or loss.

Case Study 4: Student Examination Eligibility

1. Problem Statement

Write a Python program to check whether a student is eligible to appear for an examination. Eligibility requires at least 75% attendance, completed laboratory work, at least 25 internal marks and payment of fees. Display the reasons when the student is not eligible.

2. Python Source Code

```
attendance = float(input("Enter attendance percentage: "))
lab = int(input("Lab completed? (1 for Yes, 0 for No): "))
internal = int(input("Enter internal marks: "))
fee = int(input("Fee paid? (1 for Yes, 0 for No): "))

if attendance >= 75 and lab == 1 and internal >= 25 and fee == 1:
    print("Eligible for Examination")
else:
    print("Not Eligible for Examination")
    if attendance < 75:
        print("Attendance is insufficient")
    if lab == 0:
        print("Lab work is incomplete")
    if internal < 25:
        print("Internal marks are insufficient")
    if fee == 0:
        print("Fees are not paid")
```

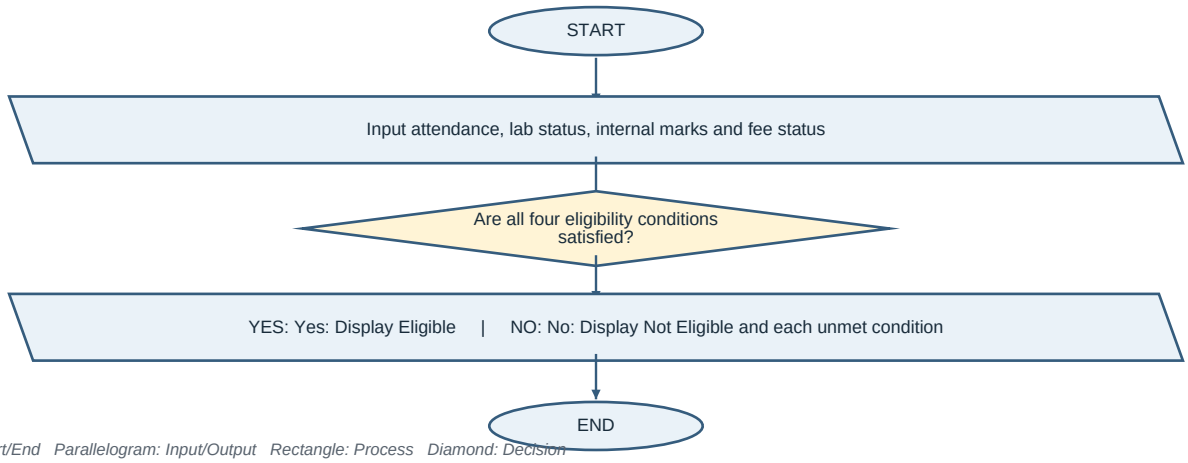
3. Sample Input

Attendance = 80, Lab completed = 1, Internal marks = 30, Fee paid = 1

4. Sample Output

```
Enter attendance percentage: 80
Lab completed? (1 for Yes, 0 for No): 1
Enter internal marks: 30
Fee paid? (1 for Yes, 0 for No): 1
Eligible for Examination
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program checks four eligibility conditions together using logical AND. If any condition is not satisfied, it displays the applicable reason or reasons.

Case Study 5: Currency Conversion and Travel Budget

1. Problem Statement

Write a Python program to convert an amount in US dollars into Indian rupees using an exchange rate. Compare the converted amount with the user's travel budget and display whether the amount is within budget or exceeds it.

2. Python Source Code

```
dollars = float(input("Enter amount in USD: "))
rate = float(input("Enter exchange rate: "))
budget = float(input("Enter travel budget in INR: "))

amount = dollars * rate

print("Amount in INR:", amount)

if amount <= budget:
    print("Status: Within Budget")
else:
    print("Status: Exceeded Budget")
    print("Extra Amount:", amount - budget)
```

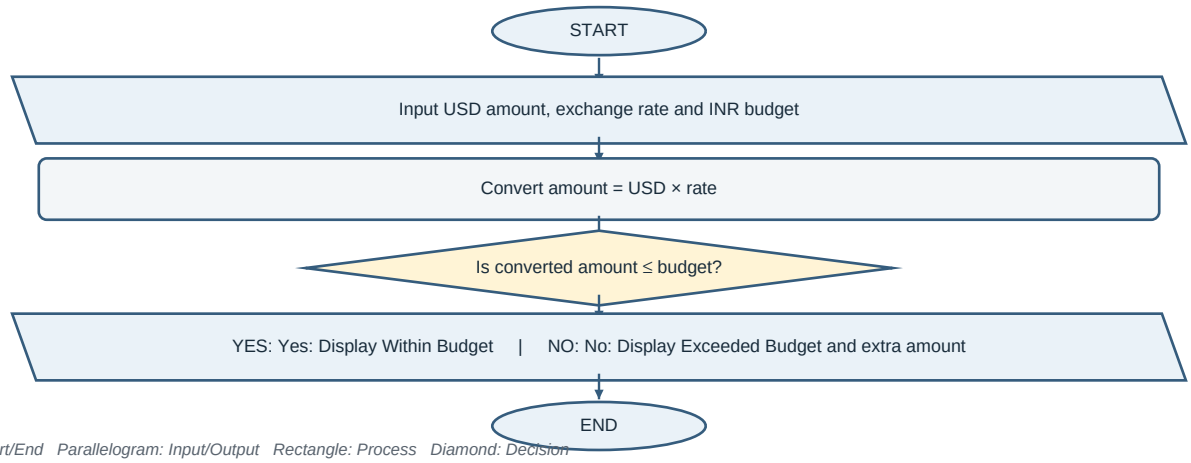
3. Sample Input

Amount = USD 100, Exchange rate = Rs. 83, Budget = Rs. 9000

4. Sample Output

```
Enter amount in USD: 100
Enter exchange rate: 83
Enter travel budget in INR: 9000
Amount in INR: 8300.0
Status: Within Budget
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program uses multiplication for currency conversion and a conditional statement to compare the converted amount with the available budget.

Case Study 6: Hospital Bill and Discount Calculation

1. Problem Statement

Write a Python program to calculate a patient's hospital bill using consultation, diagnostic and treatment fees. Give a 10% discount when the total bill is Rs. 3000 or more, and display the final bill.

2. Python Source Code

```
consultation = float(input("Enter consultation fee: "))
diagnostic = float(input("Enter diagnostic fee: "))
treatment = float(input("Enter treatment fee: "))

total = consultation + diagnostic + treatment

if total >= 3000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount

print("Total Bill:", total)
print("Discount:", discount)
print("Final Bill:", final_bill)
```

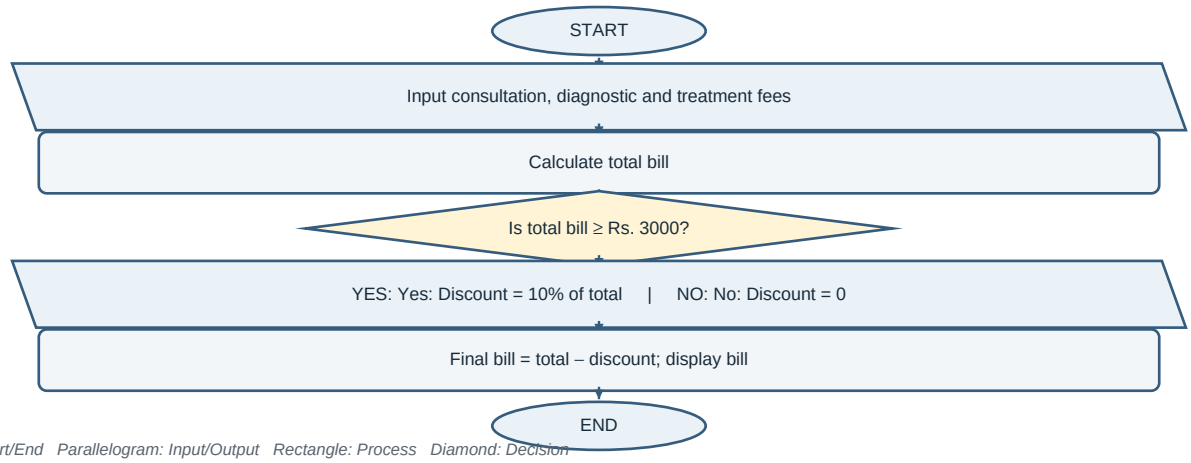
3. Sample Input

Consultation = Rs. 1000, Diagnostic = Rs. 1500, Treatment = Rs. 1000

4. Sample Output

```
Enter consultation fee: 1000
Enter diagnostic fee: 1500
Enter treatment fee: 1000
Total Bill: 3500.0
Discount: 350.0
Final Bill: 3150.0
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program adds the three service fees, checks the discount condition and subtracts the discount to calculate the final bill.

Case Study 7: Traffic Violation Fine Calculator

1. Problem Statement

Write a Python program that displays three traffic violations and accepts a choice from the user. Display the selected violation and its fine amount. Display an error message for an invalid choice.

2. Python Source Code

```
print("1. No Helmet")
print("2. Signal Jump")
print("3. Over Speeding")

choice = int(input("Enter your choice: "))

if choice == 1:
    print("Violation: No Helmet")
    print("Fine Amount: Rs.500")
elif choice == 2:
    print("Violation: Signal Jump")
    print("Fine Amount: Rs.1000")
elif choice == 3:
    print("Violation: Over Speeding")
    print("Fine Amount: Rs.2000")
else:
    print("Invalid Choice")
```

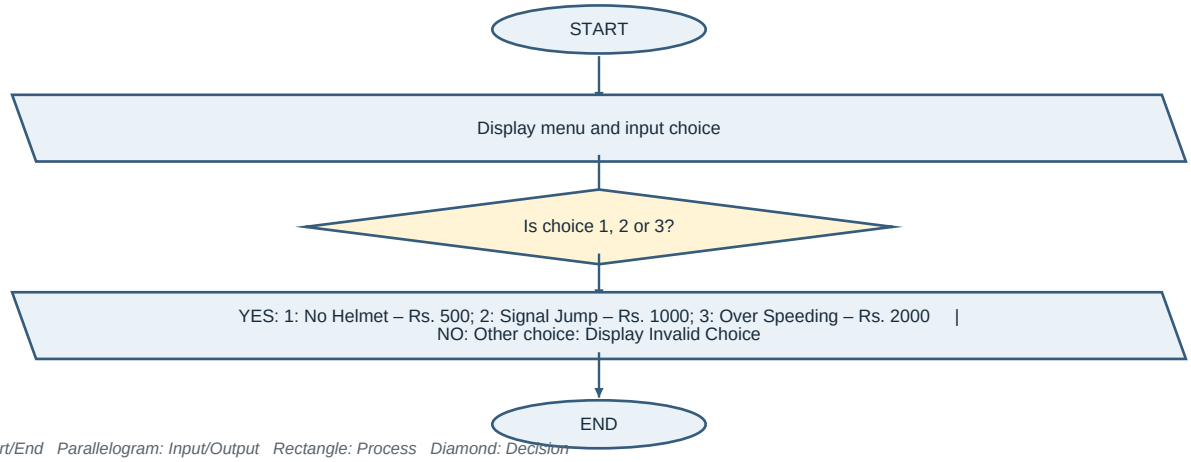
3. Sample Input

Choice = 2

4. Sample Output

```
1. No Helmet
2. Signal Jump
3. Over Speeding
Enter your choice: 2
Violation: Signal Jump
Fine Amount: Rs.1000
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program uses an if-elif-else decision structure to select the matching traffic violation and display its corresponding fine.

Case Study 8: Water Consumption Bill Calculator

1. Problem Statement

Write a Python program to calculate a water bill based on the number of units consumed. Charge Rs. 3 per unit for the first 50 units, Rs. 5 per unit for units from 51 to 100, and Rs. 8 per unit above 100 units, using cumulative slab calculation.

2. Python Source Code

```
units = int(input("Enter water consumption units: "))

if units <= 50:
    bill = units * 3
elif units <= 100:
    bill = 50 * 3 + (units - 50) * 5
else:
    bill = 50 * 3 + 50 * 5 + (units - 100) * 8

print("Water Consumption:", units)
print("Total Water Bill: Rs.", bill)
```

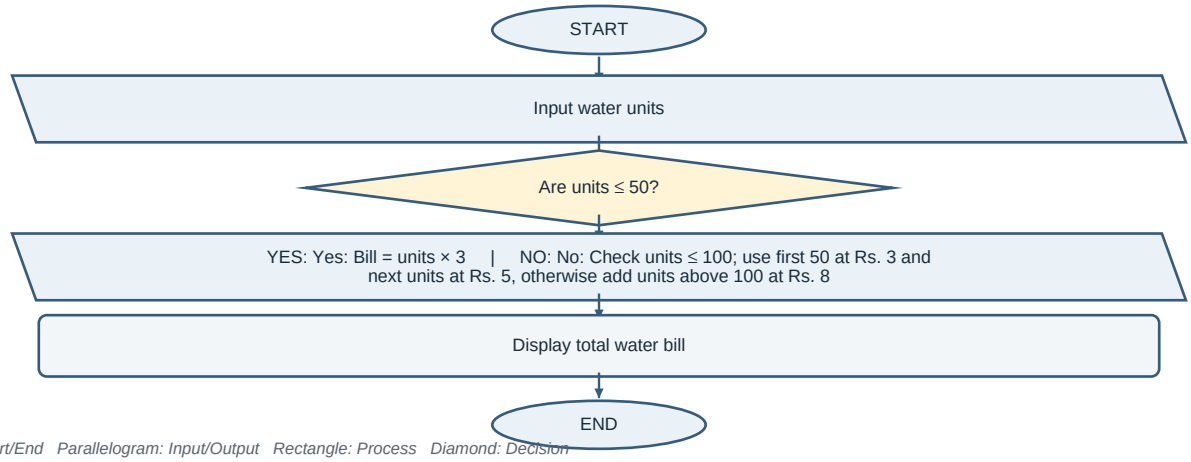
3. Sample Input

Water consumption = 120 units

4. Sample Output

```
Enter water consumption units: 120
Water Consumption: 120
Total Water Bill: Rs. 560
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program applies conditional statements to select the correct tariff slab. It calculates the bill cumulatively across all applicable slabs.

Case Study 9: Employee Salary Calculation

1. Problem Statement

Write a Python program to calculate an employee's gross salary and net salary. Gross salary is basic salary plus allowance. Net salary is gross salary minus deduction. Display the salary category as High when net salary is at least Rs. 20000; otherwise display Normal.

2. Python Source Code

```
basic_salary = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))
deduction = float(input("Enter deduction: "))

gross_salary = basic_salary + allowance
net_salary = gross_salary - deduction

print("Gross Salary:", gross_salary)
print("Net Salary:", net_salary)

if net_salary >= 20000:
    print("Salary Category: High")
else:
    print("Salary Category: Normal")
```

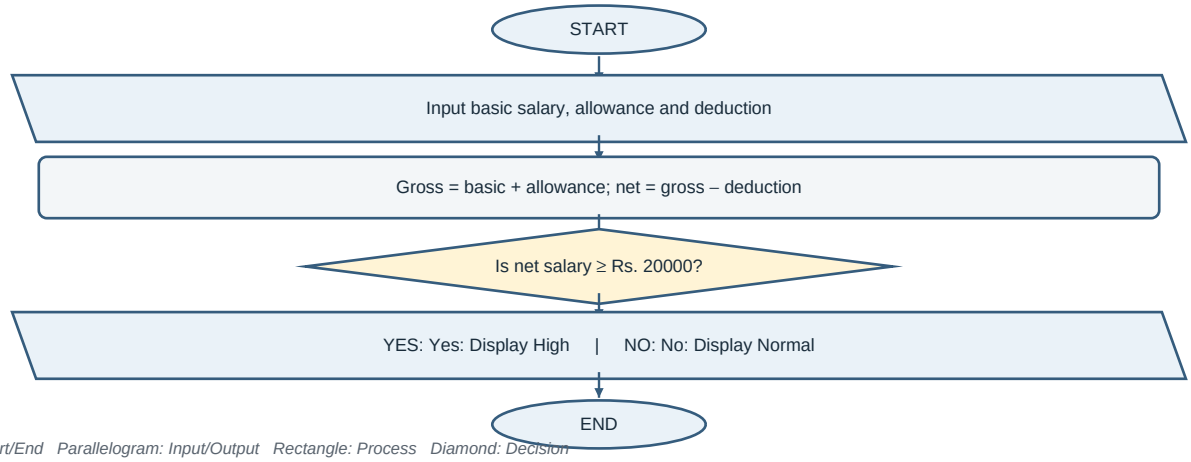
3. Sample Input

Basic salary = Rs. 25000, Allowance = Rs. 5000, Deduction = Rs. 2000

4. Sample Output

```
Enter basic salary: 25000
Enter allowance: 5000
Enter deduction: 2000
Gross Salary: 30000.0
Net Salary: 28000.0
Salary Category: High
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program calculates gross and net salary using arithmetic operators and classifies the employee's salary using a conditional statement.

Case Study 10: Attendance Percentage and Eligibility

1. Problem Statement

Write a Python program to accept the number of classes attended and the total number of classes. Validate the inputs, calculate the attendance percentage, check eligibility at 75%, and display the estimated additional classes required when the student is not eligible.

2. Python Source Code

```
attended = int(input("Enter attended classes: "))

total = int(input("Enter total classes: "))

if total > 0 and attended >= 0 and attended <= total:
    percentage = (attended / total) * 100

    print("Attendance Percentage:", percentage)

    if percentage >= 75:
        print("Eligible")
    else:
        print("Not Eligible")

        required = (75 * total - 100 * attended) / 25
        print("Minimum Additional Classes Required:", int(required) + 1)
else:
    print("Invalid Attendance Details")
```

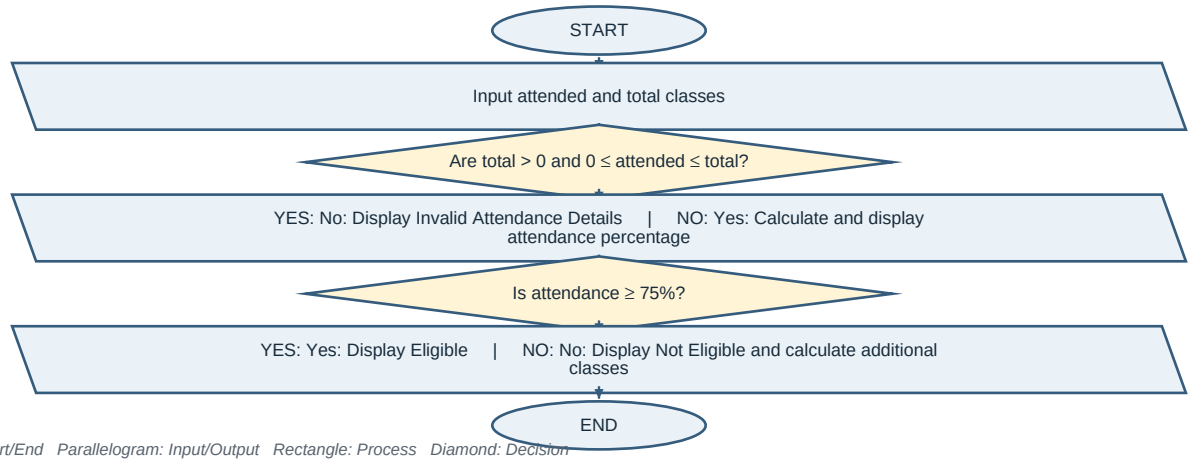
3. Sample Input

Attended classes = 60, Total classes = 100

4. Sample Output

```
Enter attended classes: 60
Enter total classes: 100
Attendance Percentage: 60.0
Not Eligible
Minimum Additional Classes Required: 61
```

5. Standard Flowchart



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

6. Conclusion

The program validates attendance details, calculates the attendance percentage and checks the 75% requirement. Note: the original expression adds 1 even when the calculated requirement is a whole number, so it may report one extra class.

Logic note: The original program prints int(required) + 1. If required is already a whole number, this adds one extra class. The source code above is kept unchanged to match the repository.

Overall Conclusion

The ten case studies demonstrate how Python can solve practical calculation and decision-making problems. They provide practice with accepting input, storing values, performing calculations, applying conditions and presenting meaningful output. The activities build a foundation for structured programming and algorithm design.

Source Repository

Python-Activity-Based-Learning — https://github.com/siddishachowdaryinapakolla-maker/Python-Activity-Based-Learning